



Codex Hackathon - Track Kits

27 back-tested, paste-ready Codex prompts • 9 tracks • 3 products each

ChatGPT Codex India Hackathon 2026
Submit at blockseblock.com

How to use this kit

Each product is on its **own page** with three things: an explicit list of **API prerequisites**, a **copy-and-paste prompt** for OpenAI Codex, and **suggested changes** to make it your own. Use the **clickable contents** on the next page to jump to any product.

Workflow: open Codex (CLI, IDE extension, or ChatGPT desktop Codex mode) in an empty folder, paste one prompt, let it build, then customise.

The boxes:

API PREREQUISITES (blue) - what you need. Every demo deploys and runs with **no login and no API key**; live/AI features are optional and clearly marked.

PROMPT (grey, monospace) - copy the whole block into Codex verbatim.

SUGGESTED CHANGES (amber) - what to change so your entry is original. Judges check originality - always customise before you submit.

Prompts are normalised to plain text for print (native scripts shown as language names, symbols as ASCII); they remain fully valid for Codex.

Contents

<u>Track 1 - Agentic Coding</u>	5
<u>Fix-It</u>	<u>5</u>
<u>TestFix</u>	<u>6</u>
<u>Refactor Radar</u>	<u>7</u>
<u>Track 2 - UX for Agentic Applications</u>	8
<u>Autonomy Dial</u>	<u>8</u>
<u>Agent Audit Trail</u>	<u>9</u>
<u>Diff Cockpit</u>	<u>10</u>
<u>Track 3 - Multimodal Intelligence</u>	11
<u>SketchSpeak</u>	<u>11</u>
<u>Point-to-Regenerate</u>	<u>12</u>
<u>Voice-to-App</u>	<u>13</u>
<u>Track 4 - Domain Agents</u>	14
<u>GST Copilot</u>	<u>14</u>
<u>ClauseClear</u>	<u>15</u>
<u>CaseBrief</u>	<u>16</u>
<u>Track 5 - Building Evals</u>	17
<u>RedGate</u>	<u>17</u>
<u>Regression Gate</u>	<u>18</u>
<u>Faithfulness Radar</u>	<u>19</u>
<u>Track 6 - AI Agents for Bharat's Businesses</u>	20
<u>Vyapari</u>	<u>20</u>
<u>Bol-Support</u>	<u>21</u>
<u>Udhaar Book</u>	<u>22</u>
<u>Track 7 - Vernacular / Indic Languages & Local Discovery</u>	23
<u>SamajhAI</u>	<u>23</u>
<u>Nagrik</u>	<u>24</u>
<u>Code Guru</u>	<u>25</u>

Track 8 - AI for Societal Good

26

<u>AccessLens</u>	<u>26</u>
<u>Alert Saathi</u>	<u>27</u>
<u>Triage Mitra</u>	<u>28</u>

Track 9 - Vibe Coding / Games / Creative

29

<u>Meme Arcade</u>	<u>29</u>
<u>Flux</u>	<u>30</u>
<u>Loregen</u>	<u>31</u>

Fix-It

Reviews a diff, writes a patch, shows a (simulated) fail->pass.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (server-side) to review any pasted diff. Note: the test-pass is simulated, not real execution on Vercel.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. I will not write any code. Don't ask questions—decide and note assumptions in the README. PRODUCT: "Fix-It" — reviews a pasted git diff like an agent (plan -> analyse -> patch -> verify) and shows its work. TRACK: Agentic Coding.

CORE FLOW: (1) paste a unified diff (bundled sample pre-filled) -> "Review & Fix"; (2) a live Agent Log streams Plan -> Analyse (bugs/security/perf/missing tests) -> Propose patch -> Run tests -> Summarise; (3) output: findings (severity P0/P1/P2, file:line, plain reason, suggested fix), the patch as a diff, a fail->pass test log, and a verdict; "Copy report (Markdown)".

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind. Deploy: Vercel. OpenAI SDK only in a server route.

PREREQUISITES (also put in README): Demo mode needs NO login and NO key. Live mode (optional) needs only OPENAI_API_KEY.

DEMO (default, no key): DEMO_MODE=true replays a canned run from /data/demo-run.json on the bundled sample diff. IMPORTANT: the "Run tests" step is a SIMULATED/heuristic result in BOTH demo and live mode — do NOT try to execute `npm test` against a pasted diff on Vercel (no checkout, ~10s function limit). State this in the README.

LIVE (optional): OPENAI_API_KEY + DEMO_MODE=false -> /api/review analyses & patches ANY pasted diff server-side; model from OPENAI_MODEL; never expose keys; .gitignore .env*; ship .env.example.

DATA: bundle a small fixture file that /data/sample-diff.txt actually targets (so the diff is concrete).

/data/demo-run.json shape: {steps:[{name,status,detail}], findings:[{severity,file,line,title,why,fix}], patch:"<unified diff>", testLog:{before,after}}.

WORK EFFICIENTLY: plan mode first (PLANS.md milestones each with an acceptance command); read files in parallel; high reasoning effort for the review/patch engine, low for UI; run app + npm test + typecheck after each milestone; /review before finishing; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "How Codex built this"); Vitest tests for the diff parser + review engine + patch applier, and a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` shows review->patch->(simulated)green-test on the sample diff with no key; npm test passes; npm run build succeeds. Then print: run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Go deep on one language's rules (Python security, Go concurrency).
- Add 'download fix as a .patch' + apply-to-pasted-repo mode.
- Connect a real public GitHub PR URL instead of pasted diffs.
- Add a severity-trend chart across multiple reviews.

TestFix

Repo with a failing test -> diagnose, fix, PR-ready diff.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (server-side). Test-pass is simulated (no repo checkout on Vercel).

PROMPT – copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "TestFix" — a repo with a failing test -> the agent diagnoses, fixes, re-runs, and produces a PR-ready diff. TRACK: Agentic Coding.

CORE FLOW: (1) load the bundled sample repo or paste files; (2) visible loop Reproduce -> Locate -> Fix -> Re-run tests -> Draft PR; (3) show the fail->pass log + the PR diff; "Copy PR (Markdown)".

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind. Deploy: Vercel. OpenAI SDK only in a server route.

PREREQUISITES (also in README): Demo = NO login/key. Live (optional) = OPENAI_API_KEY only.

DEMO (default, no key): DEMO_MODE=true replays a canned run from /data/demo-run.json on the bundled /data/sample-repo/. The test-pass is SIMULATED — do not execute tests on Vercel (see Fix-It rationale).

BUNDLE: /data/sample-repo/ = a tiny Node project using **Vitest**, with ONE seeded bug (an off-by-one in a `paginate()` util that makes `paginate.test.ts` fail); include package.json, the util, and the test.

/data/demo-run.json shape = {steps:[{name,status,detail}], diagnosis:string, patch:"<diff>", testLog:{before,after}}.

LIVE (optional): OPENAI_API_KEY + DEMO_MODE=false -> /api/fix diagnoses+patches ANY pasted files server-side; OPENAI_MODEL env; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for locate/fix, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "How Codex built this"); Vitest tests for the locate/patch logic + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install` & `npm run dev` shows reproduce->fix->(simulated)green-test->PR on the sample repo with no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Swap in your own failing-test fixture.
- Support Python (pytest) as well as JS (Vitest).
- Add an 'explain the fix for juniors' panel.
- Generate a full PR body + labels.

Refactor Radar

Scans code for dead code / anti-patterns -> refactor diffs.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) to enrich suggestions. Core scan is deterministic (ts-morph), no key.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Refactor Radar" — scans a codebase for dead code / anti-patterns / duplication and proposes each refactor as a reviewable diff with before/after (simulated) test proof. TRACK: Agentic Coding.

CORE FLOW: (1) load the bundled sample project (or paste files); (2) list issues (dead code, long functions, duplication, unused deps) with severity + location + a proposed refactor diff; (3) "Apply preview" shows before/after + a simulated test-pass; "Export refactor report".

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + **ts-morph** (AST parsing). Deploy: Vercel. LLM optional in a server route; static analysis is deterministic.

PREREQUISITES (also in README): Demo = NO login/key (deterministic AST scan). Live (optional) = OPENAI_API_KEY only, to enrich suggestions.

DEMO (default, no key): scanning uses deterministic rules in /lib/scan.ts (ts-morph) on the bundled /data/sample-project.

BUNDLE: /data/sample-project = 5 TS files containing exactly: 1 unused export, 1 duplicated function pair, 1 function >80 lines, 1 unused npm dependency in package.json, and 1 unreachable branch — so detector precision/recall is checkable.

LIVE (optional): OPENAI_API_KEY enriches wording via a server route; never required; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for the scan/refactor logic, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "How Codex built this"); Vitest tests asserting each seeded issue is detected + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` flags all 5 seeded issues with refactor diffs + before/after, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Add detectors (magic numbers, deep nesting, TODO debt).
- Point it at a real repo you paste in.
- Add an 'auto-apply safe refactors' toggle with preview.
- Track a codebase-health score over time.

Autonomy Dial

Tiered-autonomy cockpit with a live, defined trust score.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): None - fully client-side, no key ever.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior frontend engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Autonomy Dial" — supervise an AI agent with tiered autonomy: auto-approve high-confidence actions, human-review the rest, show a live trust score. TRACK: UX for Agentic Applications.

CORE FLOW: (1) a simulated agent task streams into a live timeline (Plan/Tool Call/Edit/Test/Review) with status+duration; (2) each consequential action has confidence 0–1; a user threshold auto-approves above it, queues the rest for Approve/Reject/Comment; (3) Trust Score 0–100 updates live; (4) Pause, Resume, Redirect (typed note), Kill Switch; (5) export run+decisions as JSON.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Zustand. Deploy: Vercel. NO external API.

PREREQUISITES (also in README): NONE — fully client-side, no login, no key, ever.

DATA: the run is a SIMULATED stream from /data/run.json replayed via /lib/agentStream.ts (swap this one module for a real backend stream later; document it). /data/run.json shape:

```
{steps:[{id,type:'plan|tool|edit|test|review',label,status,durationMs,confidence?:0-1,findingIds?:[]}],
findings:[{id,severity,stepId,text}]}
```

Include ~16 steps, 2 diffs, a fail->pass test, 2 findings, and a mix of confidences (some >0.9 auto-approve, some 0.4–0.7 queued).

LOGIC TO PIN DOWN: $\text{TrustScore} = \text{clamp}(100 \cdot 8 \cdot \text{overrideRate} \% 15 \cdot \text{unresolvedFindings} + 5 \cdot (\text{passedTests} > 0), 0, 100)$ (define once in /lib/trust.ts, unit-test it). Redirect only appends a logged annotation to the timeline (it does NOT alter the canned script — say so in the UI/README). Kill Switch = terminal state; Resume disabled after Kill.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for trust/threshold logic, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + how to plug a real stream + "How Codex built this"); Vitest tests for trust.ts + the stream generator + a Playwright smoke test (approve a queued action -> score changes); MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` shows the live run with threshold auto-approve, review queue, trust score, and controls, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Wire it to a real agent/LLM stream.
- Add role-based approvals (reviewer vs admin).
- Add a cost / latency / token panel.
- Make the confidence threshold per-action-type.

Agent Audit Trail

Plain-language timeline of agent actions + kill switch.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): None - fully client-side, no key.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior frontend engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Agent Audit Trail" — a plain-language timeline of everything an AI agent did, with filters and a kill-switch, for non-technical oversight. TRACK: UX for Agentic Applications.

CORE FLOW: (1) a simulated run streams into a friendly timeline — each event a card in plain English (what/why/status/time), grouped by task; (2) filters: all / decisions / external actions / errors; (3) Pause + Kill Switch; (4) "Explain this step" shows a longer canned explanation (distinct from the one-line reason); (5) export as JSON/Markdown.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Zustand. Deploy: Vercel. NO external API.

PREREQUISITES (also in README): NONE — fully client-side, no login, no key.

DATA: SIMULATED stream from /data/run.json via /lib/agentStream.ts (swappable for a real stream). /data/run.json shape: {tasks:[{taskId,title}], events:[{id,taskId,type:'decision|external|edit|error|info',reason,detail,status,timestamp}]}. Include ≥ 2 distinct tasks so "grouped by task" is real, and ≥ 1 error event.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for the event $\rightarrow$ plain-language mapper, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "How Codex built this"); Vitest tests for the mapper + stream generator + a Playwright smoke test (timeline renders, filter works, kill-switch stops it); MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` shows the plain-language trail grouped by ≥ 2 tasks with working filters + kill-switch, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Feed a real agent's logs.
- Add search + CSV/PDF export for compliance.
- Add 'flag for review' + comments per event.
- Localise the plain-language summaries.

Diff Cockpit

Risk-scores each PR hunk; approve/redirect; trust ledger.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) - rewrites risk explanations only, never the score.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior frontend engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Diff Cockpit" — review an AI agent's PR: risk-score each hunk, approve/redirect per hunk, with a trust ledger. TRACK: UX for Agentic Applications.

CORE FLOW: (1) load a bundled sample PR (multiple files/hunks); (2) each hunk gets a risk score + reason (auth/DB/secrets/deps/logic) and Approve/Reject/Comment; (3) a summary bar tracks approved vs rejected + net risk; (4) a Trust Ledger shows the agent's past accuracy from bundled history; (5) export decisions as JSON.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind. Deploy: Vercel. LLM optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (deterministic risk heuristics). Live (optional) = OPENAI_API_KEY only, to sharpen risk explanations.

DATA: /data/sample-pr.json shape:

```
{files:[{path,hunks:[{id,category:'auth|db|secrets|deps|logic',diff,riskReason}}]} — include at least one hunk per category. /data/trust-ledger.json = past runs: [{runId, predictions:[{hunkId,predictedRisk,outcome:'clean|reverted|incident'}}]; Trust "accuracy" = agreement between predictedRisk and outcome across the ledger (define in /lib/trust.ts). Do NOT invent a bare "87%" — compute it from this bundled data.
```

LOGIC: risk scoring is deterministic in /lib/risk.ts (path/keyword heuristics per category). Live mode: if OPENAI_API_KEY is set AND a "Sharpen with AI" toggle is on, /api/explain rewrites the reason text only (never changes the score); OPENAI_MODEL env; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for risk + ledger math, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "How Codex built this"); Vitest tests for the risk scorer + ledger-accuracy math + a Playwright smoke test (hunks render, approve one, summary updates); MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` shows per-hunk risk (all 5 categories present), working approve/reject, and a ledger accuracy computed from bundled history, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Add risk categories (perf, migrations, PII).
- Pull a real GitHub PR via the API.
- Add 'request changes' that drafts a review comment.
- Weight the trust ledger by recency.

SketchSpeak

Wireframe or voice -> responsive page -> refine by voice.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY with a VISION-capable model for live image->code. Browser: Web Speech (Chrome/Edge/Safari; not Firefox by default; STT needs internet).

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "SketchSpeak" — turn a wireframe image OR a spoken description into a responsive webpage, then refine it by voice. TRACK: Multimodal Intelligence.

CORE FLOW: (1) drop an image (bundled sample) OR press mic and describe (browser SpeechRecognition, default lang en-US + a language picker); (2) generate self-contained HTML with INLINED CSS, shown live in a sandboxed iframe (sandbox="allow-scripts", no allow-same-origin; in-frame CSP default-src 'none') + an editable code panel that re-renders on edit (debounced); (3) "Refine": in DEMO_MODE show 3 quick-action refinement BUTTONS (not open dictation) that map to bundled variants; in live mode accept spoken/typed refinements; (4) copy/download.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind (app UI) + CodeMirror + Web Speech API (feature-detect, degrade to text input). Deploy: Vercel. Vision LLM only in a server route.

PREREQUISITES (also in README): Demo = NO login/key. Live (optional) = OPENAI_API_KEY with a VISION-capable OPENAI_MODEL (verify the chosen model accepts image input). Web Speech caveats as noted above.

DEMO (default, no key): DEMO_MODE=true: bundled /public/sample.png -> bundled /data/demo.html; /data/refinements.json = 3 button-triggered edits -> resulting HTML.

LIVE (optional): OPENAI_API_KEY + DEMO_MODE=false -> /api/generate sends the image/description to the vision model; never expose keys; .gitignore .env*; .env.example.

CONTENT SAFETY: generated pages contain no external scripts; all styling inlined. Never eval untrusted code in the main page.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for generate/refine + iframe sandboxing, low for chrome; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + Web Speech notes + "How Codex built this"); Vitest tests for the generator/refine switch + an HTML-sanitiser + a Playwright smoke test (load sample -> preview -> apply a button refinement); .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` turns the sample wireframe into a live previewed page and applies a button refinement, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Output React/Vue components instead of plain HTML.
- Add 'match this brand palette'.
- Add an iterate-until-it-matches (render->compare) loop.
- Accept a Figma export as input.

Point-to-Regenerate

Screenshot -> code; box-select a region to regenerate.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (vision) for live regeneration. Optional webcam 'point' mode needs HTTPS + camera permission (coarse heuristic only).

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Point-to-Regenerate" — turn a screenshot into HTML, then select a region to regenerate just that component. TRACK: Multimodal Intelligence.

CORE FLOW: (1) upload a screenshot (bundled sample) -> generate a responsive page (inlined CSS) shown in a sandboxed iframe (sandbox="allow-scripts", no allow-same-origin; in-frame CSP default-src 'none'); (2) PRIMARY region-select = draw a box on the source image (deterministic coordinates); (3) "Regenerate this region" updates only that component; (4) editable code panel + copy/download.

EXPERIMENTAL (clearly labelled, live-only bonus): a webcam "point" mode using MediaPipe Tasks Vision. Be honest in the UI: MediaPipe returns hand landmarks/gestures in the CAMERA frame, so map only a coarse heuristic (dominant hand position -> left/right/top/bottom quadrant -> nearest region), NOT pixel-accurate pointing. Requires HTTPS + getUserMedia permission; show a fallback if the camera is denied/unavailable.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + CodeMirror + MediaPipe Tasks Vision (client-side, experimental mode only). Deploy: Vercel. Vision LLM only in a server route.

PREREQUISITES (also in README): Demo = NO login/key (box-draw + bundled variants). Live (optional) = OPENAI_API_KEY with a VISION-capable OPENAI_MODEL. Webcam mode needs HTTPS + camera permission. DEMO (default, no key): DEMO_MODE=true: /public/sample.png -> /data/demo.html; /data/regions.json maps named box regions -> bundled regenerated HTML. Graded acceptance uses box-draw only.

LIVE (optional): OPENAI_API_KEY + DEMO_MODE=false -> /api/generate regenerates a selected region server-side; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for region mapping + sandboxing, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + honest note on gesture accuracy + "How Codex built this"); Vitest tests for the region mapper + sanitiser + a Playwright smoke test (load sample -> box-select -> regenerate a region); .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` generates the sample page and regenerates a box-selected region, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Make box-draw multi-region.
- Target a component library (shadcn / MUI).
- Improve the gesture heuristic or use a stylus.
- Add version history of regenerations.

Voice-to-App

Speak an app idea -> working single-page app + preview.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY for live codegen. Browser: Web Speech (live mode only; feature-detect).

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Voice-to-App" — describe an app by voice and get a working single-page app with a live preview. TRACK: Multimodal Intelligence.

CORE FLOW: (1) DEMO_MODE default shows 3 spec BUTTONS whose transcripts appear "as if spoken" (mic is a live-mode-only affordance); (2) "Build it" renders a self-contained single-page app (HTML+JS, inlined CSS) live in a sandboxed iframe with sandbox="allow-scripts" (no allow-same-origin) + in-frame CSP default-src 'none' — this is REQUIRED so the generated app's own interactivity (add/remove, toggles) actually runs; (3) "Tweak" applies button-triggered canned changes in demo (spoken in live); (4) copy/download.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + CodeMirror + Web Speech API (live mode only; feature-detect). Deploy: Vercel. Codegen LLM only in a server route.

PREREQUISITES (also in README): Demo = NO login/key (3 bundled specs). Live (optional) = OPENAI_API_KEY only. Web Speech caveats as noted for this track.

DEMO (default, no key): /data/specs.json = 3 specs -> bundled generated single-file apps + 1 canned tweak each.

LIVE (optional): OPENAI_API_KEY + DEMO_MODE=false -> /api/build generates an app (bounded to a single self-contained file, no runtime npm deps) from ANY spec; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for codegen/tweak + sandboxing, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + Web Speech notes + "How Codex built this"); Vitest tests for the spec->app switch + sanitiser + a Playwright smoke test (pick a spec -> interactive app renders and works); .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` builds a working, interactive app from a bundled spec and applies a tweak, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Expand the spec grammar (auth, lists, forms).
- Persist generated apps + a gallery.
- Add one-click 'deploy this app'.
- Support a second language for the spoken spec.

GST Copilot

Reads invoices, classifies GST, drafts a return, flags issues.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) for messy free-text. Optional real data: Sandbox.co.in GST API. Avoid the NIC e-invoice sandbox - it needs a real GSTIN.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer building Indian tax-compliance software. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README. This is a drafting assistant, NOT tax/legal advice — show that in the UI. PRODUCT: "GST Copilot" — read invoices, classify GST, draft a return summary, flag mismatches, for an MSME. TRACK: Domain Agents.

CORE FLOW: (1) load bundled sample invoices (or paste rows); (2) visible pipeline: parse -> classify rate+HSN (from /data/hsn-rules.json) -> compute CGST+SGST when buyer_state == seller_state else IGST -> build a GSTR-1/3B-style summary -> flag issues (missing HSN, rate mismatch, duplicate invoice, B2B without GSTIN) with confidence; (3) summary dashboard + invoice table (row flags + fixes) + "Download filing summary (Markdown/CSV)". Low-confidence rows flagged "verify with a CA".

CONTENT SAFETY: /data/hsn-rules.json is ILLUSTRATIVE SAMPLE DATA, labelled in UI + README as "not an official rate list". Use the CURRENT post-22-Sep-2025 GST structure as the reference (slabs 5% / 18% / 40%, plus 0% exempt) — do NOT emit the old 12%/28% slabs. The optional LLM must not invent rates.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind. Deploy: Vercel. LLM optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (deterministic engines). Live (optional) = OPENAI_API_KEY (robust parsing of messy free-text). Optional real GST data: use Sandbox.co.in's free GST API — NOTE the NIC e-invoice sandbox requires a real GSTIN and is NOT viable for a keyless team.

DATA: /data/sample-invoices.json = 10 invoices incl. one missing HSN, one rate mismatch, one duplicate, one B2B w/o GSTIN, with fields {invoiceNo,date,sellerState,buyerState,lines:[{item,hsn?,qty,rate,amount}]}. /data/hsn-rules.json = {hsn: {label,gstRate}} small table.

LOGIC: gstClassify.ts, taxCompute.ts, validate.ts are deterministic + unit-tested. Live: OPENAI_API_KEY only; OPENAI_MODEL env; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for classify/compute, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "illustrative data / not tax advice" + "How Codex built this"); thorough Vitest tests for gstClassify + taxCompute (CGST/SGST vs IGST) + validate + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` classifies the sample invoices, shows the summary + all 4 flag types, exports a filing summary, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Load your own HSN table / a specific industry.
- Wire Sandbox.co.in for live e-invoice data.
- Add GSTR-2B reconciliation (purchases vs claims).
- Export a CA-ready packet (CSV + PDF).

ClauseClear

Flags risky contract/NDA clauses vs a playbook, redlines.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) to review any pasted text. Deterministic playbook engine needs no key.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer building legal-tech. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

Drafting assistant, NOT legal advice — show that in the UI.

PRODUCT: "ClauseClear" — flag risky contract/NDA clauses vs a playbook, explain plainly, propose redlines. TRACK: Domain Agents.

CORE FLOW: (1) paste a contract or load the bundled sample NDA; (2) split clauses -> match /data/playbook.json rules (liability cap, auto-renewal, IP, indemnity, governing law) -> risk score High/Med/Low -> plain-English why -> suggested redline; (3) risk summary + clause table + "Download risk memo (Markdown)". Low-confidence rows flagged "review with a lawyer".

CONTENT SAFETY: the "why" text must NOT fabricate statute sections, case names, or citations — keep reasoning to the playbook rule + plain-English logic only. Bundled data is illustrative sample.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind. Deploy: Vercel. LLM optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (deterministic heuristics in /lib/playbookEngine.ts). Live (optional) = OPENAI_API_KEY only.

DATA: /data/sample-nda.txt (a realistic sample NDA); /data/playbook.json = [{id,rule,riskIfViolated,redlineTemplate}].

LIVE (optional): OPENAI_API_KEY + DEMO_MODE=false reviews ANY pasted text vs the playbook; OPENAI_MODEL env; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for clause/playbook logic, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "not legal advice" + "How Codex built this"); Vitest tests for clause-split + playbook engine + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` analyses the sample NDA and renders the risk table + memo, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Swap the playbook for your vertical (rental, employment, vendor).
- Export DOCX with tracked changes.
- Add a 'negotiation email' generator.
- Add multi-language plain-English summaries.

CaseBrief

Fact pattern -> ranked precedents + a first-draft IRAC brief.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY + an Indian Kanoon API key for live search. Kanoon is paid/approved access (not an instant key) - demo uses a bundled fictional corpus.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer building legal-research tooling. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README. Drafting aid, NOT legal advice — show that in the UI.

PRODUCT: "CaseBrief" — fact pattern -> relevant precedents + a first-draft brief with citations. TRACK:

Domain Agents.

CORE FLOW: (1) enter a fact pattern (or load an example); (2) extract issues -> search a case corpus -> rank precedents -> extract holdings -> draft an IRAC brief (Issues/Rule/Application/Conclusion) with citations; (3) show matched cases (title, court, year, holding, relevance) + the draft; "Download brief (Markdown)".

CONTENT SAFETY (critical): the bundled /data/cases.json uses CLEARLY-FICTIONAL case names + citations, each visibly labelled "DEMO — fictional case for illustration". The brief may cite ONLY cases present in the retrieved results — it must NEVER synthesise a citation or holding not in the corpus/results. Say this in the UI + README.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind. Deploy: Vercel. LLM + external API only in a server route.

PREREQUISITES (also in README): Demo = NO login/key (deterministic ranker over the bundled corpus). Live (optional) = OPENAI_API_KEY + an Indian Kanoon API key — NOTE Indian Kanoon is a paid/approved API, NOT an instant self-serve key, so treat live mode as optional and test the demo for the hackathon.

DATA: /data/cases.json = ~20 fictional judgments {id,title(fictional),court,year,holding,keywords[]}

LIVE (optional): OPENAI_API_KEY (+ Indian Kanoon key) + DEMO_MODE=false; OPENAI_MODEL env; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for ranker + citation-extractor, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "fictional demo data / not legal advice" + "How Codex built this"); Vitest tests for the ranker + a test asserting the brief cites ONLY retrieved cases + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` returns ranked (fictional, labelled) precedents + a brief citing only those, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Replace the fictional corpus with a real public-domain set.
- Wire Indian Kanoon (approved key) for live search.
- Add a citation-graph visualisation.
- Add a 'counter-argument' section.

RedGate

Runs a test suite AND a red-team suite; scores + charts.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) for a live target. The default mock model runs everything with no key.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior engineer building an LLM evaluation + red-team tool. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "RedGate" — run a test suite AND a red-team suite against a target and score it. TRACK: Building Evals.

CORE FLOW: (1) test suite (/data/suite.json): each case = input, assertion (exact|contains|not_contains|regex|json_valid|max_latency_ms|faithfulness), expected; (2) red-team suite (/data/redteam.json): adversarial prompts each expecting refusal; (3) target = "Mock model" (default, deterministic /lib/mockModel.ts) or "Live model" (needs key); (4) "Run all" -> scoreboard (pass rate, avg latency), red-team scorecard (pass/warn/fail), per-case drawer, run-history chart, export JSON/CSV.

METRIC DEFINITIONS (implement in /lib/metrics.ts, unit-tested): faithfulness = fraction of the answer's key nouns/entities that appear in the case's provided context (0–1); red-team: FAIL = model complied with the harmful request, WARN = partial/hedged compliance, PASS = clean refusal. Pin these in tests with hand-computed expected values.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Recharts. Deploy: Vercel. Live calls only in a server route.

PREREQUISITES (also in README): Demo = NO login/key (Mock model runs everything). Live (optional) = OPENAI_API_KEY only.

DATA: /data/suite.json (10 cases across all assertion types, incl. 2 faithfulness cases with a `context` field), /data/redteam.json (6 adversarial prompts). Mock model returns deterministic outputs so scores are reproducible.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for metrics/runner, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "How Codex built this"); exhaustive Vitest tests for EVERY metric + every red-team check (hand-computed expectations) + the runner + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` runs both suites vs the mock model and shows scoreboard + red-team scorecard + chart, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Point the target at your own app's endpoint.
- Add metrics (semantic similarity, tool-call correctness).
- Expand the red-team suite (OWASP-LLM).
- Add CI / GitHub Action output.

Regression Gate

Evals every prompt change; blocks a merge on regression.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) for a live target. Mock model by default, no key.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Regression Gate" — re-run an eval suite on every prompt change, show score deltas, block a merge on regression. TRACK: Building Evals.

CORE FLOW: (1) simulated "PRs" (/data/prs.json) each changing a prompt; (2) selecting one runs /data/suite.json vs a built-in mock model, shows pass rate + per-case results + delta vs the baseline run; (3) a green "Merge allowed" / red "Blocked: regression" gate listing offending cases; (4) run-history chart + export.

REGRESSION RULE (define in /lib/regression.ts, unit-tested): BLOCKED if pass-rate drops > 0 points vs baseline OR any previously-passing case now fails. State the rule in the UI.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Recharts. Deploy: Vercel. Live model optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (mock model). Live (optional) = OPENAI_API_KEY only.

DATA: /data/prs.json = [{id,title,promptBefore,promptAfter}] (>=3, incl. one that regresses); /data/suite.json shared with the mock model.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for metrics/delta/gate, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + how to wire real GitHub Actions + "How Codex built this"); Vitest tests for metrics + delta + gate + a Playwright smoke test (select regressing PR -> "Blocked"); .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` runs a simulated PR and shows a correct pass/block gate with deltas, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Wire a real GitHub Action + PR comments.
- Tune the regression rule (allow a small tolerance).
- Add per-metric gates.
- Add a Slack/webhook alert on block.

Faithfulness Radar

Scores a RAG agent (faithfulness/hallucination) + drift chart.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) for LLM-as-judge on uploaded datasets. Deterministic scorers need no key.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Faithfulness Radar" — score a RAG agent for faithfulness/hallucination/relevancy/context-precision and chart drift. TRACK: Building Evals.

CORE FLOW: (1) bundled /data/rag.json (question, context[], answer, groundTruth); (2) "Score" computes the 4 metrics with the DETERMINISTIC heuristics below; (3) dashboard + per-question drill-down + an over-time drift chart across saved runs; (4) export JSON/CSV.

HEURISTIC DEFINITIONS (implement in /lib/metrics.ts, unit-tested against hand-computed values):

- faithfulness = fraction of answer sentences whose key nouns/entities all appear in context[] (0–1);
- hallucinationRate = 1 - faithfulness;
- answerRelevancy = token-overlap (Jaccard) between question and answer;
- contextPrecision = fraction of context[] chunks that share >=1 key entity with groundTruth.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Recharts. Deploy: Vercel. Live LLM-judge optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (deterministic scorers). Live (optional) = OPENAI_API_KEY (LLM-as-judge on uploaded datasets).

DATA: /data/rag.json = ~8 items incl. 2 deliberately unfaithful answers so the metrics visibly differ.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for the scorers, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + the exact metric formulas + "How Codex built this"); Vitest tests per metric pinned to hand-computed expected values + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` scores the sample dataset (unfaithful items score lower) and shows a drift chart, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Plug in your own RAG dataset (upload CSV/JSON).
- Add an LLM-judge mode and compare vs the heuristic.
- Add per-source attribution scoring.
- Alert when drift crosses a threshold.

Vyapari

WhatsApp-style invoicing + fuzzy UPI reconciliation + GST.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) for robust parsing. A REAL WhatsApp version needs WhatsApp Cloud API (business verification ~2-7 days); Indic voice via Bhashini (PoC tier, not free for prod). Browser: Web Speech.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer building for Indian MSMEs. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Vyapari" — a WhatsApp-style assistant for a kirana shop: plain-language invoicing, UPI reconciliation, GST reminders, bilingual + Hindi voice. TRACK: AI Agents for Bharat's Businesses.

CORE FLOW (mobile-first chat UI): (1) invoice by message: parse "2 kg cheeni 90, 1 Parle-G 10, chai 20" via the grammar below -> invoice + total + shareable summary -> Open Invoices; (2) reconcile: paste a UPI/bank SMS -> fuzzy-match to an open invoice via the formula below -> mark paid or ask to confirm; (3) ledger: "Who owes me" (+ages), "Send GST reminder" (drafts a message — TEXT only, no rate math), weekly sales total; (4) English/Hindi toggle for UI + generated messages.

INVOICE GRAMMAR (deterministic /lib/invoiceParser.ts): ``<qty><unit> <item> <price>`` repeated, comma-separated; unit {kg,g,l,ml,pc,pkt,dozen}; if a token can't be parsed, flag it for manual entry (don't guess). Scope parsing to Hinglish/Latin script; Hindi-voice transcripts are transliterated to Latin before parsing (note this).

RECONCILE (deterministic /lib/reconcile.ts): $\text{score} = 0.5 * \text{amountExact} + 0.3 * \text{JaroWinkler}(\text{payerName}) + 0.2 * \text{keywordOverlap}(\text{narration}, \text{invoice})$; ≥ 0.8 auto-match, $0.5 - 0.8$ ask-to-confirm, < 0.5 no match.

Unit-test all three bands.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Web Speech API (feature-detect).

Deploy: Vercel. Optional LLM only in a server route.

PREREQUISITES (also in README): Demo = NO login/key (deterministic parser+reconcile, localStorage, seeded data). Live (optional) = OPENAI_API_KEY only. A REAL WhatsApp version needs the WhatsApp Cloud API (business verification takes ~2–7 business days) and, for Indic voice, Bhashini (PoC/non-production tier only, not free for production) — NEITHER is used in this demo; this is a web/PWA that mimics WhatsApp UX with zero external deps.

DATA: /data/seed.json = products + 3 open invoices + 3 realistic bank-SMS strings in 2 bank formats incl. one truncated/ambiguous, e.g. "Rs.200.00 credited to a/c XX1234 on 30-07-26 from VPA parleg@okaxis (Ramesh Kumar) UPI Ref 123456789012" and a terser HDFC-style one.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for parser + fuzzy reconcile, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + Web Speech notes + "How Codex built this"); Vitest tests for invoiceParser + reconcile (all 3 bands, incl. the ambiguous SMS) + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible + bilingual; product original.

DONE WHEN: `npm install` & `npm run dev` creates an invoice by message and reconciles the clean + ambiguous payments correctly, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Wire a real WhatsApp Cloud API number (start verification day 1).
- Add Hindi voice via Bhashini.
- Add an ONDC catalog / storefront.
- Add expense tracking + a daily summary.

Bol-Support

Bilingual support agent (orders/FAQ/returns) + handoff.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY and/or Bhashini (PoC tier) for real intent + translation. A real channel needs WhatsApp Cloud API (verification days).

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Bol-Support" — a bilingual WhatsApp-style support agent (order status / FAQ / returns) with human handoff. TRACK: AI Agents for Bharat's Businesses.

CORE FLOW: (1) customer messages in English or Hindi; a reply-language TOGGLE controls output language while input is free-form (auto-handled); (2) deterministic intent classifier (order status / FAQ / return / other) answers from seeded orders+FAQ; (3) low-confidence or "other" -> a "handed to human" state shown in an agent-side inbox that PERSISTS in localStorage across reloads.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind. Deploy: Vercel. LLM/translation optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (deterministic over seeded data). Live (optional) = OPENAI_API_KEY and/or Bhashini (PoC-tier only) for real intent+translation; a real WhatsApp channel needs WhatsApp Cloud API (verification ~2–7 business days). None used in the demo.

DATA: /data/seed.json = orders + FAQs + canned English/Hindi replies + labelled intent examples.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for the intent classifier + handoff rule, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + Bhashini/WhatsApp swap notes + "How Codex built this"); Vitest tests for the classifier + handoff rule + a Playwright smoke test (order-status reply; trigger handoff persists); .env.example; MIT LICENSE; mobile-first + accessible + bilingual; product original.

DONE WHEN: `npm install && npm run dev` answers order-status + an FAQ in both languages and persists a handoff across reload, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Add intents (warranty, address change) + your FAQ.
- Wire Bhashini for real translation.
- Add a live agent-takeover UI.
- Capture CSAT after resolution.

Udhaar Book

Credit ledger with tone-matched reminders + cashflow.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) to personalise reminder wording.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Udhaar Book" — a credit ledger for a small shop: track udhaar, draft tone-matched reminders, show cashflow. TRACK: AI Agents for Bharat's Businesses.

CORE FLOW: (1) add customers + credit/repayment entries (seeded); (2) "Who owes me" (amounts + ages + status on-time/overdue); (3) "Draft reminder" — tone-matched, bilingual, per the rule below; (4) cashflow view (receivables, this-week collections, trend). Persists in localStorage.

TONE RULE (deterministic /lib/reminder.ts): "repeat-late" = 2+ reminders in the last 90 days OR currently >30 days overdue -> firmer template; else gentle template. A reply-language toggle controls output language.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind. Deploy: Vercel. LLM optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key. Live (optional) = OPENAI_API_KEY only (personalise wording).

DATA: /data/seed.json = customers + entries covering >=6 weeks of history so the cashflow "trend" chart is visibly populated; include >=1 on-time and >=1 repeat-late customer.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for ledger math + tone selection, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "How Codex built this"); Vitest tests for ledger math + tone selection + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible + bilingual; product original.

DONE WHEN: `npm install && npm run dev` tracks credit, lists who owes me with correct tone reminders, and shows a populated cashflow trend, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Add SMS/WhatsApp send for reminders.
- Add interest / aging buckets.
- Add a customer credit-limit + risk score.
- Add a monthly statement export.

SamajhAI

Explains a coding error in your Indian language, with voice.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional; Bhashini/IndicTrans2 swap documented). Browser: Web Speech + getVoices() check per language.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer building an Indic-language dev tool. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "SamajhAI" — explain a coding error/concept in the user's Indian language, plainly, with a fix, read aloud. TRACK: Vernacular / Indic Languages.

CORE FLOW: (1) paste an error (or use the mic, language-aware); (2) pick a language: English, Hindi, Tamil, Bengali, Telugu, Marathi; (3) output in that language: What it means / Why / How to fix (with a corrected snippet) + a "Speak" button; toggle original <-> explanation.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Web Speech API (feature-detect). Deploy: Vercel. LLM only in a server route.

PREREQUISITES (also in README): Demo = NO login/key (bundled pre-written explanations). Live (optional) = OPENAI_API_KEY (README shows how to swap in Bhashini/IndicTrans2 for translation; Bhashini is PoC-tier). Web Speech caveats as above.

DEMO (default, no key): DEMO_MODE=true ships /data/samples.json = 5 common errors x all 6 languages (pre-written). Unmatched input -> the "not in demo set" state, never a silent LLM call.

SPEAK: before enabling "Speak", check speechSynthesis.getVoices() for the chosen lang; if none, show "no TTS voice for this language on this device".

LIVE (optional): OPENAI_API_KEY + DEMO_MODE=false handles ANY input/language; OPENAI_MODEL env; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for lookup/language routing, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + Web Speech notes + "have a native speaker check the 30 bundled strings" + "How Codex built this"); Vitest tests for sample lookup + language routing + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible; correct fonts for all scripts; product original.

DONE WHEN: `npm install && npm run dev` explains a bundled error in Hindi + one other language with a working getVoices()-guarded Speak, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Add more languages / a romanized-script toggle.
- Swap in Bhashini/IndicTrans2 for live translation.
- Add a 'translate my code comments' mode.
- Target a specific curriculum / board.

Nagrik

Voice civic-query agent: govt schemes, answered in-language.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY and/or Bhashini (PoC tier) for open queries + wider languages. Browser: Web Speech.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README. This is an informational demo, NOT official government advice — show that in the UI.

PRODUCT: "Nagrik" — ask about a govt scheme by voice in your language; get a clear spoken answer from a curated FAQ. TRACK: Vernacular / Indic Languages.

CORE FLOW: (1) pick a language (Hindi / Tamil / Bengali / English), mic or type; (2) match to a curated FAQ (/data/schemes.json: eligibility, documents, how-to-apply) with a real "source" citation baked into the data; (3) answer in the chosen language, shown + spoken; (4) suggested follow-ups. Unmatched query -> "not covered by this demo's FAQ set" (never improvise a scheme answer).

CONTENT SAFETY: answers come ONLY from the bundled FAQ; the optional LLM may rephrase/translate but must not invent eligibility/documents. Visible disclaimer: "informational demo — verify on the official government portal."

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Web Speech API (feature-detect).

Deploy: Vercel. LLM/translation optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key. Live (optional) = OPENAI_API_KEY and/or Bhashini (PoC-tier). Web Speech caveats + getVoices() guard as for this track.

DATA: /data/schemes.json = 6 schemes {name, source(url/portal ref), eligibility, documents[], howToApply} with pre-written answers in all listed languages.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for FAQ matching/routing, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + disclaimer + Web Speech notes + "How Codex built this"); Vitest tests for the FAQ matcher + language routing + a Playwright smoke test (ask -> answer renders + speaks); .env.example; MIT LICENSE; mobile-first + accessible; correct fonts; product original.

DONE WHEN: `npm install && npm run dev` answers a bundled scheme question in Hindi + one other language with speech + a source line, and shows the "no match" state for an unknown query, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Load real scheme FAQs for your state.
- Wire Bhashini voice for more languages.
- Add a document-checklist generator + reminders.
- Add WhatsApp delivery.

Code Guru

Vernacular coding tutor: concept + runnable example + voice.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY and/or IndicTrans2 (optional). Browser: Web Speech. Runs example code in a sandboxed iframe/Worker.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Code Guru" — teach a programming concept step-by-step in the learner's Indian language, with a runnable example + audio. TRACK: Vernacular / Indic Languages.

CORE FLOW: (1) pick a concept (bundled: variables, loops, functions, async, recursion) + language (Hindi / Tamil / Bengali / English); (2) step-by-step explanation in that language + a runnable code example + "Speak"; (3) a "check understanding" mini-quiz; (4) romanized-script toggle. Unmatched concept in demo -> "not in the demo set".

SAFE CODE RUNNER (required): run example JS ONLY inside an iframe with sandbox="allow-scripts" (no allow-same-origin) OR a Web Worker with no DOM/network; NEVER eval/new Function in the main page; add a watchdog timeout to kill infinite loops.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Web Speech API + the sandboxed runner above. Deploy: Vercel. LLM/translation optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (bundled pre-translated lessons). Live (optional) = OPENAI_API_KEY and/or IndicTrans2. Web Speech caveats + getVoices() guard as for this track.

DATA: /data/lessons.json = 5 concepts x all listed languages, each {explanationSteps[], exampleCode, quiz:[{q,options,answerIndex}]}

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for lesson routing + the safe runner, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + Web Speech notes + "How Codex built this"); Vitest tests for lesson lookup + quiz scoring + a Playwright smoke test (pick concept+language -> lesson + runnable example render); .env.example; MIT LICENSE; mobile-first + accessible; correct fonts; product original.

DONE WHEN: `npm install && npm run dev` teaches a concept in Hindi + one other language with a sandboxed runnable example + getVoices()-guarded Speak, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Add more concepts / a full track.
- Add a Python runner (Pyodide) alongside JS.
- Add spaced-repetition of the quizzes.
- Localise to more languages.

AccessLens

One-click WCAG auditor (axe-core) + fixes + alt text + score.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) for richer alt text. axe-core runs fully client-side, no key.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior frontend engineer building an accessibility tool. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "AccessLens" — find WCAG issues, explain the fixes, draft alt text, score the page. TRACK: AI for Societal Good.

CORE FLOW: (1) paste an HTML snippet (default) or load the bundled inaccessible sample (live-URL fetch needs a server proxy — note it, don't promise it); (2) run axe-core; (3) issues grouped by severity with element (selector+snippet), plain-English "why", and corrected markup; (4) draft alt text for missing it; (5) score 0–100 + "Download report (Markdown)" + "Re-run".

CORRECT SANDBOX PATTERN (required): parse the pasted HTML with DOMParser (never executes scripts on parse), render it into an iframe created with sandbox="allow-scripts" (NOT allow-same-origin, NOT allow-popups) so axe-core can execute inside it to compute contrast/layout, and inject an in-frame CSP `default-src 'none'` so untrusted markup can't call out. axe-core is injected only into that isolated frame. Do NOT say "scripts disabled" — axe-core must run inside the audited context.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + axe-core (client-side). Deploy: Vercel. Optional LLM (alt text) only in a server route.

PREREQUISITES (also in README): Demo = NO login/key (axe-core runs in-browser). Live (optional) = OPENAI_API_KEY only, for richer alt text (a deterministic filename/context heuristic is the default).

Tag any AI-drafted alt text "AI-suggested — verify" with an edit box.

DATA: /public/sample.html = a deliberately inaccessible page (missing alts, low contrast, no labels, bad heading order).

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for the audit wiring + report/score, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + "How Codex built this"); Vitest tests for report + score + alt-text heuristic + a Playwright smoke test (load sample -> issues + score render); .env.example; MIT LICENSE; mobile-first + accessible (the tool itself must pass a11y); product original.

DONE WHEN: `npm install && npm run dev` audits the bundled page (axe-core executes in the sandboxed frame) and shows grouped issues + score + report, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Batch-audit multiple pages/URLs (server proxy).
- Add a colour-contrast fixer that suggests palettes.
- Add a CI mode (JSON output + fail thresholds).
- Add mobile-a11y checks.

Alert Saathi

Multilingual disaster-alert triage on a map.

API PREREQUISITES

Deploy / live demo: None - bundled alerts + free OSM tiles (keep the required '(c) OpenStreetMap contributors' attribution).

Live mode (optional): OPENAI_API_KEY + a data.gov.in key + IMD access (IMD needs onboarding + IP-whitelisting) for live data.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior frontend engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README. Informational demo, NOT an official alert source — show that in the UI.

PRODUCT: "Alert Saathi" — convert disaster/weather alerts into plain, actionable, multilingual messages on a map. TRACK: AI for Societal Good.

CORE FLOW: (1) a feed of bundled sample alerts (flood/heatwave/cyclone) with location + severity; (2) each alert -> plain language + recommended actions (from a bundled per-hazard-type table) + translation, as a card; (3) a Leaflet map plots alerts by severity; (4) filter by area/severity/language; "Copy alert".

CONTENT SAFETY (critical): the LLM/translation layer may ONLY rephrase + translate the alert's existing fields (location, severity, hazard, official text) — it must NOT add, infer, or embellish facts.

"Recommended actions" come ONLY from a bundled /data/actions.json lookup per hazard type (e.g., flood -> move to higher ground), never free-generated. Persistent disclaimer: "For info/demo only — follow official IMD/NDMA and local-authority instructions."

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Leaflet (OpenStreetMap tiles).

Deploy: Vercel. LLM/translation optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (bundled alerts + OSM tiles; keep the required "© OpenStreetMap contributors" attribution visible; heavy prod traffic should move to a paid tile provider). Live (optional) = OPENAI_API_KEY + a data.gov.in API key + IMD access — NOTE IMD's API needs authenticated onboarding + IP-whitelisting and data.gov.in needs a registered key, so treat live data as optional.

DATA: /data/alerts.json = 6 alerts {id,hazard,severity,lat,lng,area,officialText,translations{}};

/data/actions.json = hazard -> recommended actions[].

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for the transform/severity/filter logic, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + OSM attribution + IMD/data.gov.in friction + disclaimer + "How Codex built this"); Vitest tests for severity + alert-transform (asserts actions come from the table, not invented) + a Playwright smoke test (alerts + map render, filter works); .env.example; MIT LICENSE; mobile-first + accessible + multilingual; product original.

DONE WHEN: `npm install && npm run dev` shows simplified, translated alerts (actions from the bundled table) on a working attributed map with filters, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Wire live data.gov.in / IMD feeds (plan onboarding early).
- Add SMS/WhatsApp broadcast of alerts.
- Add crowd-report intake + verification.
- Add per-pincode subscriptions.

Triage Mitra

Rules-based symptom triage with strong 'not a diagnosis' framing.

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) - phrases conversation only; the rules engine always decides. No medical API required.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README. This is NOT a diagnostic or medical tool — show that prominently.

PRODUCT: "Triage Mitra" — ask about symptoms, suggest a triage level (self-care / see a clinic / emergency), in the user's language. TRACK: AI for Societal Good.

CORE FLOW: (1) a guided chat asks primary symptom + red-flag questions (bundled rules /data/triage.json); (2) output a triage level + reasoning + safe next steps; red-flag answers ALWAYS show emergency guidance; (3) language toggle + optional voice; (4) a persistent, unmissable disclaimer + "call local emergency services" prompt for emergencies.

SAFETY (critical): the DETERMINISTIC rules engine (/lib/triage.ts) ALWAYS decides the level; the optional LLM only phrases conversation and can NEVER change which red-flags fire or the level. Base /data/triage.json on a real, citable public-health framework (e.g., WHO IMCI / CDC self-triage / NHS-111-style) and cite it in the README; every ambiguous or uncovered case defaults to the MORE cautious tier ("see a clinic"), never "self-care". State that these rules are a hackathon simplification, not production clinical logic.

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Web Speech API (optional). Deploy: Vercel. LLM optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (rules engine). Live (optional) = OPENAI_API_KEY only (phrasing). No medical API required.

DATA: /data/triage.json = symptom -> questions -> level rules incl. explicit red-flag -> emergency mappings; cite the source framework in a comment.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for triage rules + red-flag safety, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + a clear safety/disclaimer + cited framework + "How Codex built this"); thorough Vitest tests covering EVERY red-flag -> emergency path + the cautious-default rule + a Playwright smoke test; .env.example; MIT LICENSE; mobile-first + accessible + multilingual; product original.

DONE WHEN: `npm install && npm run dev` runs the symptom flow, returns a level with disclaimers, always escalates red-flags, and defaults ambiguous cases to "see a clinic", no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Expand rules from a cited framework (WHO/CDC/MoHFW).
- Add regional-language voice.
- Add a 'find nearest clinic' (Maps) step.
- Add a caregiver mode for children/elderly.

Meme Arcade

Upload an image -> it becomes the sprite in a juicy game.

API PREREQUISITES

Deploy / live demo: None - single static file, runs offline, no login/key/backend.

Live mode (optional): None.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior game developer. Build a COMPLETE, polished, original browser game in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Meme Arcade" — upload an image -> it becomes the player sprite in a juicy, replayable game.

TRACK: Vibe Coding / Games / Creative.

GAME DESIGN: (1) upload/pick an image (bundled default) -> auto-crop/scale to a sprite (via `<input type=file> + URL.createObjectURL + drawImage` — works offline); (2) a mechanic picker with 3 one-input mechanics, each with CONCRETE rules: Dodge (obstacles spawn from the right every ``max(400, 900 score*4)`` ms; tap to jump; AABB collision), Flap (tap to flap vs gravity, gap-pipes), Gravity-Flip (tap flips vertical gravity, ceiling/floor gaps); combo grows on a near-miss (player within 12px of an obstacle without collision), resets on hit; (3) particles + screen-shake + trail; procedural WebAudio SFX + a simple `OscillatorNode` bassline (no files); (4) screens: Title (Tap to start + best), Playing (score+combo HUD), Game Over (score, best, Tap to retry, copyable share text). Touch + keyboard; best score in `localStorage`; pause on blur.

TECH: prefer a SINGLE self-contained `index.html` (inline CSS + vanilla JS + Canvas 2D + WebAudio) that runs by double-clicking (`file:///`) with NO backend/assets/build/network. (If you use a helper lib, do NOT — pure Canvas keeps the "open the file" promise; a bundler/served-page requirement would break it.) `AudioContext` must be created/resumed on the first tap (Title screen) per browser autoplay rules.

PREREQUISITES (also in README): NONE — no login, no key, no backend, no network. Works offline.

WORK EFFICIENTLY: short PLANS.md first; build in milestones (playable core -> sprite-from-image -> mechanics -> juice -> audio -> screens -> mobile), open/run after each; /review before finishing; small commits `git commit --author="Codex <codex@openai.com>"`.

DELIVER: `index.html` (whole game, JS in clear commented sections: `input/physics/spawn/render/audio/state`); `AGENTS.md`; README (play + 3-min static Vercel deploy + how to reskin + "How Codex built this"); `vercel.json` (static); MIT LICENSE.

DONE WHEN: opening `index.html` gives a fully playable, juicy game where an uploaded image is the sprite, across all 3 mechanics, with sound (resumed on first tap), on mouse/keyboard/touch, and a saved best score, no console errors. Then print: run locally, static Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Reskin the theme (festival, cricket, campus).
- Add a new mechanic or a boss.
- Add a daily-seed challenge + local leaderboard.
- Let players share a generated game link.

Flux

One-tap gravity-flip endless runner (single file, offline).

API PREREQUISITES

Deploy / live demo: None - single static file, runs offline, no login/key/backend.

Live mode (optional): None.

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior game developer. Build a COMPLETE, polished, original single-file browser game in the current empty folder. No code from me; don't ask questions—note assumptions in the README.

PRODUCT: "Flux" — a one-tap gravity-flip endless runner. TRACK: Vibe Coding / Games / Creative.

GAME DESIGN: character auto-runs at constant x-speed; one input (tap/click/Space) flips vertical gravity floor<->ceiling to dodge obstacles (gaps that tighten with score); difficulty ramp: obstacle gap = $\text{max}(120, 260 \text{ score} * 1.5)$ px, spawn spacing shrinks with score; combo grows on a near-miss (within 14px, no collision), resets on hit. Juice: particles on flip/near-miss, subtle screen shake on hit, trail.

Audio: procedural WebAudio SFX + a looping OscillatorNode bassline (no files); AudioContext resumed on first tap. Screens: Title (Tap to start + best), Playing (score+combo HUD), Game Over (score, best, Tap to retry, copyable share text). Touch + keyboard; best in localStorage; pause on blur.

TECH: ONE self-contained index.html (inline CSS + vanilla JS + Canvas 2D + WebAudio); NO backend/assets/build/network; runs by double-clicking; 60fps via requestAnimationFrame; crisp at any devicePixelRatio; colour-blind-friendly palette.

PREREQUISITES (also in README): NONE — no login, key, backend, or network. Offline.

WORK EFFICIENTLY: short PLANS.md first; milestones (core -> juice -> audio -> screens -> mobile), open/run after each; /review before finishing; small commits git commit -author="Codex <codex@openai.com>".

DELIVER: index.html (commented sections); AGENTS.md; README (play + 3-min static deploy + reskin + "How Codex built this"); vercel.json (static); MIT LICENSE.

DONE WHEN: opening index.html gives a fully playable, juicy gravity-flip game with the ramp + combo above, sound (resumed on first tap), on mouse/keyboard/touch, saved best score, no console errors. Then print: run locally, static Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Reskin (space, monsoon, neon).
- Add power-ups / obstacle variety.
- Add levels or a zen mode.
- Add a daily-seed challenge.

Loregen

AI dungeon-master mini-adventure (Next.js app, not single-file).

API PREREQUISITES

Deploy / live demo: None - deploys and runs the full demo with NO login and NO API key.

Live mode (optional): OPENAI_API_KEY (optional) for open-ended live scenes. Note: this one is a Next.js app (needs npm install / build).

PROMPT — copy & paste into Codex

You are OpenAI Codex, a senior full-stack engineer. Build a COMPLETE, deployable web app in the current empty folder. No code from me; don't ask questions—note assumptions in the README. NOTE: unlike the other two Track 9 products, this ships as a Next.js app (optional server route), NOT a single-file game — so it needs npm install / dev / Vercel build.

PRODUCT: "Loregen" — a 10–15 min AI dungeon-master text adventure with light visuals + a shareable recap. TRACK: Vibe Coding / Games / Creative.

CORE FLOW: (1) pick a theme (bundled: haunted temple / space derelict / monsoon city); (2) a narrated scene with 2–4 choices that branch via a client-side story graph; simple inventory + health; light Canvas visuals per scene; (3) an ending screen with a copyable recap.

STORY GRAPH (define + validate): /data/story.json nodes = {id, text, art?, choices:[{label, nextId, effects?:{inventory?:string[], health?:number}}], ending?:boolean}. Require >=3 themes, each with >=10 nodes and >=2 distinct endings, so "graph with multiple endings" is real (unit-test reachability of all endings).

STACK: Next.js (App Router, latest stable) + TypeScript + Tailwind + Canvas (pick Canvas, not p5.js, for the light visuals). Deploy: Vercel. LLM optional in a server route.

PREREQUISITES (also in README): Demo = NO login/key (bundled branching story). Live (optional) = OPENAI_API_KEY only (open-ended scenes from theme+state).

LIVE (optional): OPENAI_API_KEY + DEMO_MODE=false; OPENAI_MODEL env; never expose keys; .gitignore .env*; .env.example.

WORK EFFICIENTLY: plan mode first (PLANS.md w/ acceptance commands); parallel reads; high effort for the story-graph engine + state, low for UI; app+npm test+typecheck each milestone; /review before done; small commits git commit --author="Codex <codex@openai.com>".

DELIVER: full code; AGENTS.md; README (run + 5-min Vercel deploy + PREREQUISITES + how to add a story + "How Codex built this"); Vitest tests for the story-graph engine + inventory/state + ending-reachability + a Playwright smoke test (start -> choices -> an ending); .env.example; MIT LICENSE; mobile-first + accessible; product original.

DONE WHEN: `npm install && npm run dev` plays a full branching run to an ending with visuals + a shareable recap, no key; npm test passes; npm run build succeeds. Then print run commands, Vercel deploy steps, a 5-line demo-video script.

SUGGESTED CHANGES (make it yours)

- Write your own themed story graph.
- Add art per scene (generated or drawn).
- Add save/continue + multiple slots.
- Add an LLM live mode for open-ended play.